MTH 4300
Spring 2026

Programming Problem Set 4: Object-Oriented Programming - `Robogame`

Directions: Download the template files provided on Brightspace. Submit your source code via Gradescope, in `.cpp` and `.h` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Working with AI on This Assignment

This assignment has a structured AI component. Follow these steps in order.

Step 1: Write your code on your own first. Give each part a genuine attempt before using any AI. If your code compiles and runs but gives wrong answers, try tracing through it with a specific input on paper before moving on to Step 2.

Step 2: If you're stuck, use AI as a debugging tutor. Don't paste your code and ask "what's wrong." Instead, use a prompt like this:

> *"I'm writing a C++ class called Robogame that [describe what it should do]. Here is my code. I'm getting [wrong output / crash / compiler error]. Don't fix my code or show me corrected code. Instead, pick a specific line and ask me what I think a particular variable or array element contains at that moment. Help me find the bug by making me trace through my own logic. Keep a back-and-forth conversation."*

If you use this step, save the conversation — you'll submit a link to it.

Step 3: After your code passes the autograder, do a code review conversation. The goal is to test whether you can explain what you wrote. Paste one of the prompts below along with your working code into a new AI conversation (ChatGPT, Claude, Copilot, etc.). The AI will quiz you — treat it like a study session. This is your first time separating code into a .h and .cpp file, with more moving parts than previous assignments. Walking through your decisions out loud is a good way to check that you can explain how the pieces connect.

Paste the prompt below along with all your working code (robot.h, robot.cpp, and main.cpp) into a new AI conversation:

> *"Here is my working C++ code for a Robogame class (robot.h, robot.cpp) and simulation (main.cpp). Act as a technical interviewer. Ask me questions one at a time about my design and implementation choices. Start easier and build toward harder questions. If my answer is vague or wrong, don't correct me — ask a follow-up that helps me see the issue. Do 5–6 rounds. Then propose a small modification to my class and ask me to implement it. Critique my implementation."*

Submit a link to the conversation as a comment at the top of your `.cpp` file, along with any Step 2 conversation links.

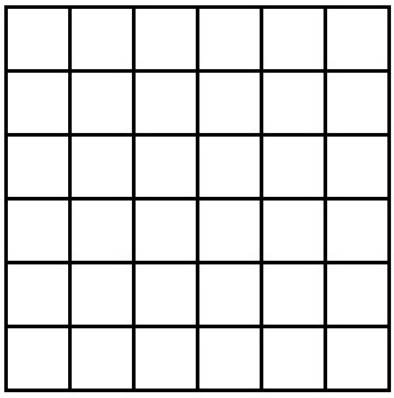
Be sure to read the SPECIFICATIONS carefully! And write comments!

General Specifications and Notes:

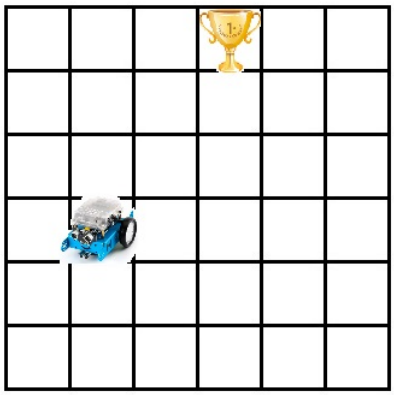
- No global variables should be used.
- The only headers allowed (in any of your files) are `iostream`, `random`, `"robot.h"`.
- You cannot use vectors or any advanced data structure that can store an arbitrarily long collection of integers, other than built-in C++ arrays.
- There is no user input in either part.
- Remember to use `const` whenever appropriate.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

Consider the following 6×6 grid:



Suppose I drop a robot and a prize on two random squares:



I then play a game: I let the robot move randomly. It moves one square up, down, to the left, or the right, one move at a time. I let the robot move until it reaches the prize or moves to a square off the grid.

Question: what is the probability that the robot reaches the prize before it moves off the grid? Answer this question by simulating 50,000 games and writing a class as follows.

4a) `Robogame` **class (robot.h and robot.cpp)**

Create a class called `Robogame`, whose objects will each represent a game. The attributes for each `Robogame` object will represent the locations of the robot and the prize. You will write methods that allow the game to play out.

Specifically, you should implement your class according to the following specifications. I suggest you deal with the methods in the order I wrote them. You are also free to add additional attributes and methods if you wish.

Your class definition and declarations should go in robot.h; the implementations should go in robot.cpp.

Each object in the class should have the following **attribute variables**:

- `robot_`, an array containing two `int`s, represents the square currently containing the robot. (The first element of the array corresponds to the row number, and the second is the column.)
- `prize_`, an array containing two `int`s between 0 and 5, represents the square that contains the prize. (The first element of the array corresponds to the row number, and the second is the column.)

The class should also contain the following **methods**:

1. A constructor, used to create a game, should receive no arguments. The constructor should initialize both `robot_` and `prize_` randomly, each as an array containing two `int`s between 0 and 5, inclusive. (They could be initialized to the same square, in which case the game will end immediately.) I have included a helper function in the template file to help you generate random numbers.
2. Four accessor methods: `get_robot_row()`, `get_robot_col()`, `get_prize_row()`, `get_prize_col()`, which return the row or column from the attributes appropriately.
3. `set_robot()`, which should receive two `int`s `i` and `j`, and return nothing. This should set the location of `this->robot_` to be `{i, j}`. (You may not use this method very much, but it's helpful for testing.)
4. `set_prize()`, which should receive two `int`s `i` and `j`, and return nothing. This should set the location of `this->prize_` to be `{i, j}`. (You *almost certainly* will not use this method very much, but again, it's helpful for testing.)

MTH 4500 S26

5. `display()`, which should receive no outside arguments and return nothing. This should print out a grid representation of the game in its current state: it should print six lines, each with six characters – a `P` for the square with a prize, an `R` for the square with the robot, or a `.` for any of the other squares. For example, `display()` could produce

```
...P..
......
......
.R....
......
......
```

   if `this->robot_` was `{3,1}` and `this->prize_` was `{0,3}`. If the prize and robot coincide, that square should contain a `C`; if the robot's position is off the grid, it should not be depicted.

6. `win()`, which should receive no outside arguments, and return `true` if the current values in `this->robot_` equal the values of `this->prize_`; the function should return `false` otherwise.

7. `off_grid()`, which should receive no outside arguments, and return `true` if the current value of `this->robot_` has either entry not between `0` and `5`, inclusive; the function should return `false` otherwise.

8. `step()`, which should receive no outside arguments and return nothing. This function should choose a random direction from {up, down, left, right} and change the robot's position by one square in the chosen direction. (Each direction should be equally likely to be selected.) Use the helper function in the template file to help with the random number generation.

> *Hint: Build this class one component at a time, in the order listed above. After implementing each method, test it before moving on. The template file includes test code you can use — uncomment the relevant portions as you go.*

4b) **Simulation client code (main.cpp)**

Remember that the attributes should *only be used in the class definition – not in your client code*!

Answer the question we asked initially. In client code, simulate 50,000 different `Robogame`s (create a new object for each simulation!). Have the game's robot take random steps until either the robot lands on the square with the prize or lands on a square outside the grid. The program should count how many games out of the 50,000 end with the robot finding the prize. You should probably be using the `win()`, `off_grid()` and `step()` methods.

You will submit this part in a separate submission area on Gradescope, *without* your robot.cpp and robot.h files (*only* main.cpp should be submitted for this part).

Specifications: your programs must

- contain a definition of the class `Robogame` that includes the 11 methods listed above, which allows my test code to run correctly, with the class declaration in robot.h and the implementations in robot.cpp. (*Both* robot.h and robot.cpp should be uploaded to the same submission on Gradescope. You can do this by selecting both and dragging and dropping them.)
- contain code that uses the robot class to simulate 50,000 games and outputs the percentage of the games where the robot reaches the prize before reaching a square off the grid.
- not contain any references to `robot_` or `prize_` in client code (i.e., outside of the class definition).

The autograder will call your class methods directly and check the results. Part (b) will be checked manually.

If your code doesn't follow specifications, you may lose additional credit even if it passes the test cases on Gradescope.

4c) **AI Code Review + Practice Questions**

AI Code Review: After your code passes the autograder, complete the AI code review conversation described at the beginning of this assignment. Submit a link to the conversation as a comment at the top of your `.cpp` file. See the "Working with AI on This Assignment" box on page 1 for the prompt and full instructions.

The following are extra practice questions for you. They are good preparation for exams. Try them on your own, and let me know if you have questions.

1. Write some code and run it to help you answer this question: In the games that the robot wins, how many steps, on average, does it take to win? Report what you find and where you made a change in your code.
2. Suppose the game had two prizes, and the robot wins after it finds *both*. Which parts of your code would need to be modified, and which would stay the same? Explain and state the changes you would make.
3. Suppose the game changed so the robot cannot visit any square twice. The game ends with a win when the robot gets the prize, or a loss when the robot wanders off the grid or cannot move. Which parts of your code would need to be modified, and which would stay the same? Explain and state the changes you would make. (Assume you are still only allowed the same headers as previously stated.)

MTH 4300 S26